bSQUARE
The Mobile & Embedded Systems Experts

Hobart Corporation



HLX Scale

Software Architecture Document

10/06/2010

Revision: 1.0

Submitted by:

BSQUARE Corporation
110 110th Ave NE Suite 200
Bellevue, WA 98004
(425) 519-5900 (Voice)
(425) 519-5999 (Fax)

To:

Hobart Corporation
701 S. Ridge Ave
Troy, OH 45374

Attn: Debra Dyer
Telephone #

IMPORTANT: This document is intended only for the use of the individual or entity to which it is addressed, and contains information, which is confidential and proprietary. If the reader of this document is not the intended recipient, or the employee or agent responsible for delivering the document to the intended recipient, you are hereby notified that any dissemination, distribution or copying of this document is strictly prohibited. If you have received this document in error, please notify the sender immediately and return the original document.

Project Name	HLX Scale Linux Port
Document ID	
Author(s)	Minhaj Ahmed Sr. Software Engineer II Bsquare Professional Engineering Services (PES) 2100 Lakeside Blvd. St. 450 Richardson, TX 75080
Reviewed by	Michael Browne Director of Engineering 2100 Lakeside Blvd , Suite 450 Richardson, TX 75082
Approved by:	Bsquare Project Manager: John Imboden Sr. Program Manager 3480 West Market Street, Suite 105 Fairlawn, OH 44333 Client Project Manager: Name Title Division
Delivery Date	

Revision History

Revision	Date	By	Comments
0.1	12/29/2009	M. Ahmed	Initial draft
1.0	10/06/2010	M. Ahmed	Updated based on Hobart's feedback.

Contents

1 INTRODUCTION 4
1.1 DOCUMENT PURPOSE 4
1.2 DEFINITIONS, ACRONYMS, AND ABBREVIATION 5
1.3 APPLICABLE DOCUMENTS 5
2 OVERVIEW 6
3 MESSAGING ARCHITECTURE 6
3.1 MESSAGE STREAM 6
4 THREADING AND SYNCHRONIZATION 9
4.1 THREADING 9
4.2 SYNCHRONIZATION 11
5 DEVICE MANAGEMENT 12
6 APPENDIX 15
6.1 NAME SPACE USED 15

FIGURES

Figure 1.1 Messaging Stream Class Hierarchy 6

Figure1. 2 Accessing the messaging stream at CMSGSafeStream level 7

Figure.1. 3Accessing the message stream at CSemStream level 7

Figure.1. 4 Layering of the Messaging Architecture. 8

Figure 1.5 Consumer Thread 9

Figure1. 6 Class Usage 10

Figure 1.7 Synchronization primitive class hierarchy 11

Figure 1.8: USBManager hierarchical diagram 12

Figure 1.9 RealTimeController hierarchical diagram 12

Figure 1.10 Hobart printer attachment sequence 14

1 Introduction

Hobart is the world leader in commercial food equipment and services industries. It manufactures products for warewashing and waste handling; food preparation; baking; cooking; weighing, wrapping and labeling.
The HLX service scale allows Hobart's line of scales to provide food retail management with a new level of control. The HLX integrates Microsoft Windows® components to provide enhanced ease of use, increased productivity and real-time connection to headquarter operations. HLX was engineered with the Windows XP-embedded operating system (XPe) in order to provide easy access to a wide range of programs, such as Internet Explorer, Windows Media Player and Outlook Express. Managers can easily access the HLX service scale from multiple locations. Its remote-access technology allows users to make changes to the system on the fly or to evaluate operator activities. The HLX scale also allows users to attach multiple external devices such as printers and keyboards and to add and/or upgrade software.

Following are some features supported by HLX service scale:

- Programmable Flashkeys for markdowns, product navigation and searches.
- Unlimited text length for product and ingredient descriptions on the scale.
- E-mail alerts when the scale is out of labels or the cassette is left open.
- Three levels of frequent-shopper discounting.
- Four graphic images per PLU for enhanced merchandising.
- Database synchronization with other department scales.
- Internal electronic logging of service calls.

1.1 Document Purpose

This document captures the basic building blocks of overall architecture of the HLX Scale application suite (HLX Suite). It is intended for developers and QA engineers in order to facilitate a better understanding of the existing code and provides ease of future enhancements. This document may be used in conjunction with the Bsquare HLX Porting Guide as references for porting the XPe code to a Linux based platform.

Bsquare will port the Hobart HLX Scale application software from the Microsoft Windows XPe environment to the Linux environment running on an AMD LX800 processor while maintaining multiple OS support with a common code base.

1.2 Definitions, Acronyms, and Abbreviation

The following terms and acronyms are used throughout this document.

Bsquare	Bsquare Corporation
XPe	Windows XP Embedded
FIFO	First In First Out
OS	Operating System
PnP	Plug and Play
WDK	Microsoft Windows Driver Kit
UI	User Interface
HLX Suites	HLX Scale application suite
API	Application Programmer's Interface

1.3 Applicable Documents

The following table contains documents that were referenced during the creation of this document.

Ref. #	Document Name	Document ID/Location
1	Hobart Peripheral Attachment Sequence Diagram.docx	http://anarchy/Restricted2006/Hobart_Linux_Port/trunk/Docs
2	Overview of scale software architecture.doc	http://bportal/PES/Program%20Management%20Office/Project%20Documents/Forms/AllItems.aspx?RootFolder=http%3a%2f%2fbportal%2fPES%2fProgram%20Management%20Office%2fProject%20Documents%2fHobart%20%2d%2083322&FolderCTID=0x01200092446C2034BF074094EADD0BA1E7BAAD

Table 1 – Applicable Documents

2 Overview

The HLX Scale application suite (HLX Suite) consists of three main components, the Web Server, the User Interface (UI) and the Scale Service. These components interact with each other through various classes and interfaces to provide a complete user experience.

The HLX Suite uses an entirely browser based UI which interacts with the GoAhead Web server to provide the HLX user interface. This setup facilitates both local and remote access to the scale service allowing store personnel and Hobart service technicians to connect with the scale when necessary.

Scale Service provided the business logic for the device. Scale Service is responsible for communicating data to/from the UI, reading and writing data to the database and interfacing with the scale's peripherals. Scale Service is a Windows service consisting of multiple threads which communicate with each other via a messaging infrastructure based on publish-subscribe architecture. At its core, this infrastructure relies on Win32 based IPC mechanisms like critical sections, semaphores, and events.

In the overall scheme, the UI receives web pages (the static part of the presentation) and graphical images from the GoAhead web server and the data (the dynamic part) from the Scale Service via a socket connection managed by a separate Java applet.

3 Messaging Architecture

As described in the overview section above various threads within the Scale Service communicate with each other through a messaging frame work. This frame work forms the most basic and important component of the HLX Suite and is the fundamental building block of the inter-thread communication architecture.

Hobart messaging scheme can be divided into two basic components. The first component consists of classes which provide basic synchronized streaming operation and the second component forms the interface which Scale Service uses for communication.

3.1 Message Stream

At the most basic level the streaming mechanism consists of three classes. CMSGStream, CMSGSafeStream and CSemStream reference Figure 1.1.

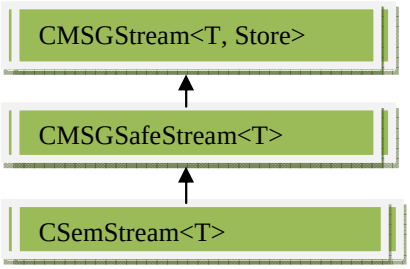


Figure 1.1 Messaging Stream Class Hierarchy

All three classes are template classes with CMSGStream forming the base class. CMSGStream accepts two (2) types of parameters; the first defines the type of message the stream handles and the second is an optional user defined class that provides basic storage and stream manipulation APIs. If no type parameter is provided for the second variable then the class defaults to std::deque class for providing the basic streaming functionality. All user

provided type parameters must conform to the basic stream manipulation APIs of the std::deque class in order for the architecture to work properly.

CMSGStream is inherited by CMSGSafeStream template class. This class uses HLX synchronization architecture to provide synchronized access to the stream. All pop, push and clear type APIs are called under the protection of a critical section. This architecture allows two threads communicating via a stream to have safe accesses to it.

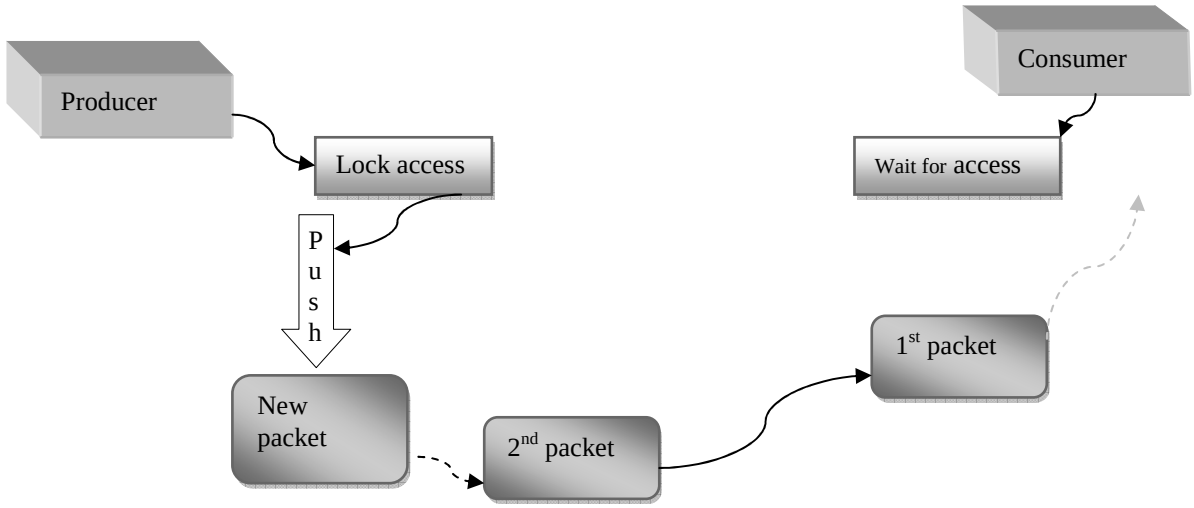


Figure1. 2 Accessing the messaging stream at CMSGSafeStream level

The final class providing the basic streaming functionality in the HLX Suite is CSemStream. This class is critical in providing consumer/producer behavior of the messaging architecture with notification mechanism built intrinsically in the scheme. A semaphore managed by HLX Synchronization Architecture is used for this purpose. Producer in this scheme can produce a defined maximum number of messages before blocking for the consumer to consume some of the already queued messages. The consumer on the other hand waits on a message to be pushed onto the stream at which point it removes the messages and proceeds with its normal processing.

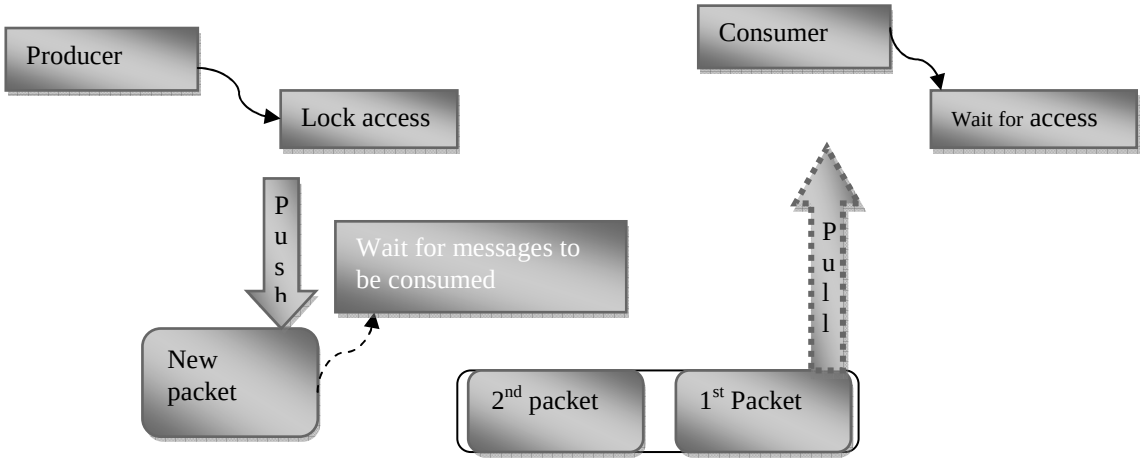


Figure.1. 3Accessing the message stream at CSemStream level

In order to provide a high level interface to the streaming architecture, the HLX Suite provides APIs which can be called by the applications. These APIs along with the lower level streaming architecture discussed above provide the complete interface to the inter-thread communication architecture of the Scale Service. The main class in this scheme is CMessageCenter. It operates in an independent CTLSThread class based thread. CMessageCenter builds a message map by using std::multimap sorted associative container class. The first type parameter of this class is the message ID that a consumer is interested in receiving. The second parameter is a std::pair class consisting of a pointer to the consumer and a Boolean specifying whether the consumer is to be removed from the message map after receiving the first message with the specified message ID. Methods for registering, deregistering and setting up notifications are part of the exported APIs of the class.

CMessageCenter operates in an independent CTLSThread class based thread. The main loop waits on the messages to be pushed onto the stream at which point it dispatches it onto a consumer(s) based on the message map.

CEvolMsgReceiver class builds on the interface exported by CMessageCenter and implements APIs which are instance specific. This allows for consumers to make use of the CMessageCenter services which are specific to their instance. CEvolMsgReciever receives all messages via a CMessageCheckThread based receiver thread. This thread retrieves messages from the stream and passes them on to the receiver for processing.

In addition to the above main classes HLX Suites implement message facilitator and proxy classes which allow for serialization and management of messages. These classes include EvolMessage, EvolMessageWrapper, MessageReceiverProxy and ProxyMessage see Figure 1.4.

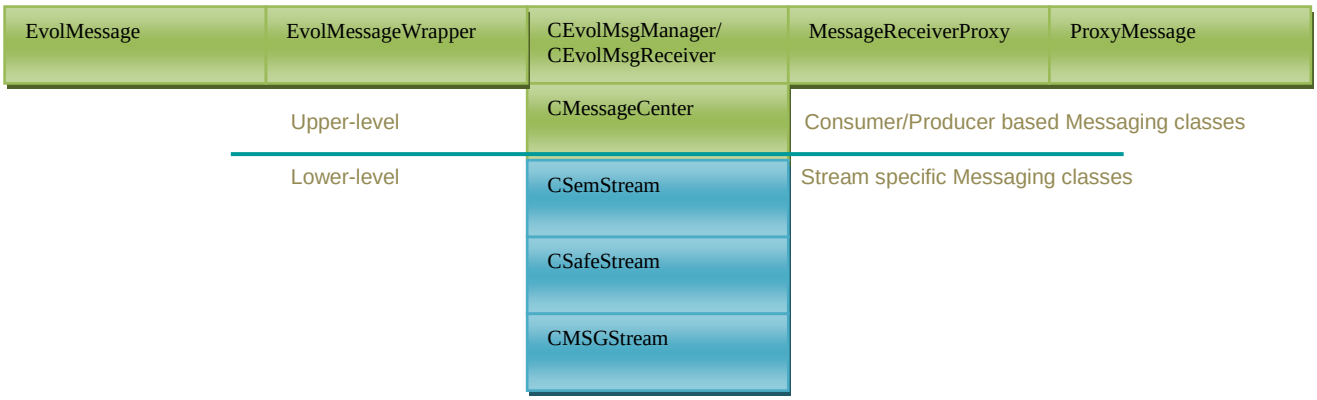


Figure.1. 4 Layering of the Messaging Architecture.

```
CSemStream<int> incoming_stream;

//
//get the handle to the semaphore
//
semaphore = incoming_stream.GetNewMessageSemaphore();

//
//wait for a message to arrive
//
If (WAIT_OBJECT_0 == semaphore.Dec())
{
        //
        //retrieve the message
        //
        int p;
        incoming_stream >> p;
}
```

Figure 1.5 Consumer Thread

4 Threading and Synchronization

This section details thread creation and management interface along with inter-thread synchronization APIs.

4.1 Threading

CRunnableClass, CTLSThread, TLSHandle, and CThreadTerminator are the main classes which form the threading architecture for the HLX Suite. CRunnableClass is an abstract class which provides a basic interface to classes requiring an independent thread execution. All thread based classes must inherit from this class. While the actual creation and management of thread takes place in CTLSThread class, CRunnableClass implements the interface for clean exiting of a thread by use of an event.

CRunnableClass works hand-in-hand with CTLSThread to provide thread creation, management and execution interface. CTLSThread constructor accepts instance of a given CRunnableClass derived class and creates a Win32 based thread. If required the main execution loop of the CTLSThread registers the thread with the Session Manager before calling derived class based execution loop for continued processing. Figure 1.6 shows CTLSThread's main loop. It is important to note here that CTLSThread's destructor uses CRunnableClass's exit scheme to terminate the thread before returning thus any CRunnableClass based thread would terminate upon CTLSThread object going out of scope.

```
unsigned int __stdcall CTLSThread::STATIC_Run(void *pCTLSThread)
{
	try
	{
		CTLSThread *thread = (CTLSThread *) pCTLSThread;
		DWORD result = 0;
		try
		{
			if (thread->m_managedThread)
				SessionManager::getInstance()->registerThread(thread->m_ThreadID, thread->m_superThread);
			CRunnableClass *runnable = &thread->m_runnable;

			TheTLSHandle.SetValue( runnable->m_ExitEvent.GetHandle() );

			thread->m_state = Running;
			result = runnable->Run();

			if (thread->m_managedThread)
				SessionManager::getInstance()->deRegisterThread(thread->m_ThreadID, thread-
>m_superThread);
		}
		catch (Util::CTerminateNow&)
		{
			if (thread->m_managedThread)
				SessionManager::getInstance()->deRegisterThread(thread->m_ThreadID, thread-
>m_superThread);
		}
		catch (...)
		{
			if (thread->m_managedThread)
				SessionManager::getInstance()->deRegisterThread(thread->m_ThreadID, thread-
>m_superThread);
		}
		thread->m_state = Exited;
		_endthreadex ( result );
		return 0;
	}

	catch (...)
	{
		return 0;
	}
}
```

Figure1. 6 Class Usage

The HLX Suite's threading architecture supports thread specific memory allocation using TLSHandle class. TLSHandle only allocates a single 32 bit size memory per thread. Threads requiring additional memory must allocate it on the heap and save the pointer in the thread specific memory managed by TLSHandle. CTLSThread class however uses this memory to save the event handle used to signal thread termination to CRunnableClass derived classes.

Finally, CThreadTerminator class is a very simple interface implemented by the architecture to terminate the thread. The class calls Win32 based TerminateThread at instantiation. This class is primarily used by the HLX Suite in the try catch construct. TerminateThread is thrown when an exception occurs allowing the exception handler to manage the error without having to provide for thread termination logic.

4.2 Synchronization

For mutual-exclusion synchronization between threads of the same process, the HLX Suites implements classes to manage the critical section. Two important classes in this scheme are CSmarCriticalSection and CLockCriticalSetion. While CLockCriticalSection provides the basic API for entering and leaving critical section CSmartCriticalSection serves as the abstraction layer for platform dependent calls.

In addition to the critical section specific synchronization classes HLX Suite uses CMSGSemaphore, CMSGMutex, CMSGEvent, and Mutex classes for implementing the basic synchronization primitive and signaling interface. Each of these class helps manage an appropriate handle. CMSGMutex, CMSGEvent, and CMSGSemaphore each inherits from CSmartHandle which provides generic handle management APIs for the HLX Suite.

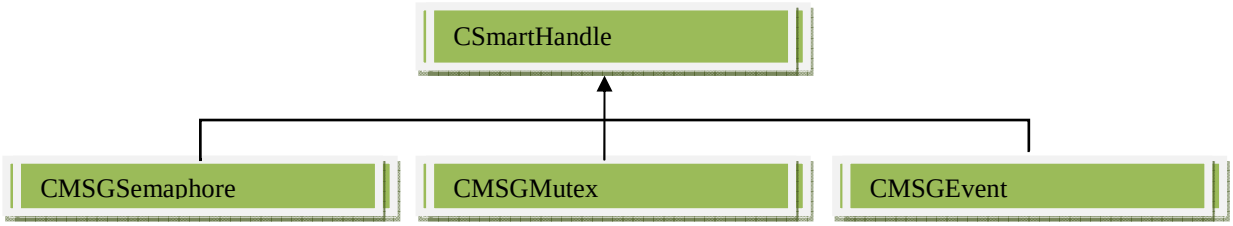


Figure 1.7 Synchronization primitive class hierarchy

CMSGSemaphore class is used by the Messaging Architecture and provides the semaphore handling interface. CMSGSemaphore exported APIs implement platform dependent increment and decrement operations for the semaphore. Decrement implementation uses global util::MSGWaitForSingleObject routine to throw an exception if the thread terminates before a relevant event occurs. Much like CMSGSemaphore class CMSGMutex class serves as an abstraction to Win32 based mutex APIs. It exports basic mutex handling interface for acquiring and releasing the mutex.

CMSGEvent class is used to create event handle. It inherits from CSmartHandle class. Overloaded constructors of CMSGEvent can be used to pass in platform specific handle, CSmartHandle or CMSGEvent handle. The rest of the routines serve as wrapper to platform specific event handling APIs. Additionally, the synchronization architecture provides CResetEvent class for manual event implementation. It has only the constructor and the destructor and depends on CMSGEvent class to either reset (in the constructor) or set (in the destructor) the event.

Class Mutex implements platform specific mutex. It uses ThreadingMode as a template variable. This variable is either set to a Win32 (Win32Threads) or VxWorks specific header. Code in Win32Threads is simply a wrapper around the underlying win32 thread APIs and serves as abstraction for platform portability. Mutex class is used with Lock template class which simply acquires and releases the mutex. Appropriate flag maintains the state of the lock. Mutex is released in destructor hence every time the Mutex object goes out of scope the mutex is released.

5 Device Management

The HLX Suites structure for the device interface architecture is divided into user and kernel level components. Only the user level components are covered here as the details of the kernel level components are beyond the scope of this document.

At the user level the overall architecture can be further divided into two major components. The first component is responsible for opening communication channel with the kernel level driver, managing device attachment and detachment and initiating device specific threads. The second component includes the communication structure and the device specific event handling classes. USBManager, RealTimeController and CUSBCommBase are the main classes responsible for implementing the Device Management architecture for the HLX Suite.

USBManager, RealTimeController and AT91Boot_DLL form the first component of the Device Management architecture. USBManager inherits from CHobartTimer and CEvolMsgReceiver (Figure 1.8) and makes use of the Messaging architecture to communicate device specific events. It maintains a USB descriptor map which associates each USB node id with its corresponding device descriptor. A timer member routine initiated from requestScanForAPCAtBootUp method is used to scan for attached devices. Firmware is loaded into the devices needing a firmware download using the AT91Boot_DLL interface. Once a device is ready USBManager initiates the RealTimeController to handle device specific thread initialization.

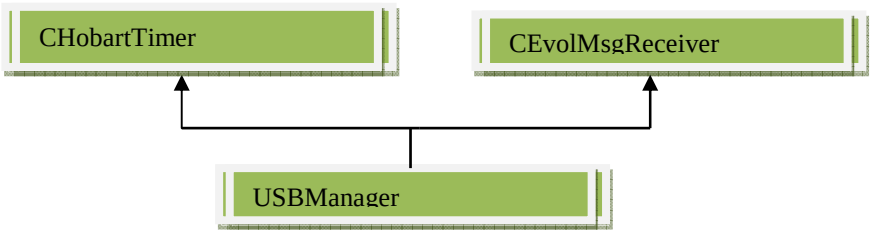


Figure 1.8: USBManager hierarchical diagram

RealTimeController inherits from CEvolMsgReceiver (Figure 1.9) and much like USBManager relies on MessageCenter for inter-thread communication. Upon a device attach event RealTimeController instantiates the appropriate Device Communication thread. All device specific communication threads inherit from CUSBCommBase class and form the back born of the communication structure of the device management architecture and are discussed in more details in the paragraphs below.

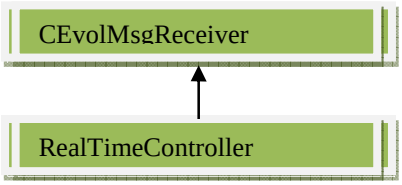


Figure 1.9 RealTimeController hierarchical diagram

CUSBCommBase class uses CUsbPipe class to implement USB pipe for the HLX Suite. It exports APIs that other elements of the device management architecture may use in order to establish USB bulk and interrupt based communication with the device(s). Each CUsbPipe instance corresponds to an endpoint on the USB device. The constructor of the class uses the descriptor of a given device to extract relevant information like the direction and the type of endpoint being communicated to. Win32 based CreateFile is then used to establish the communication channel with the kernel driver. Each CUsbPipe based pipe resides in an independent CTLSThread based thread

which CUSBCommBase class manages through exported APIs. The main loop of the CUsbPipe thread manages all the USB bulk transfers in addition to dispatching all communication received on an USB In Pipe to an appropriate CUSBCommBase derived Device Communication class.

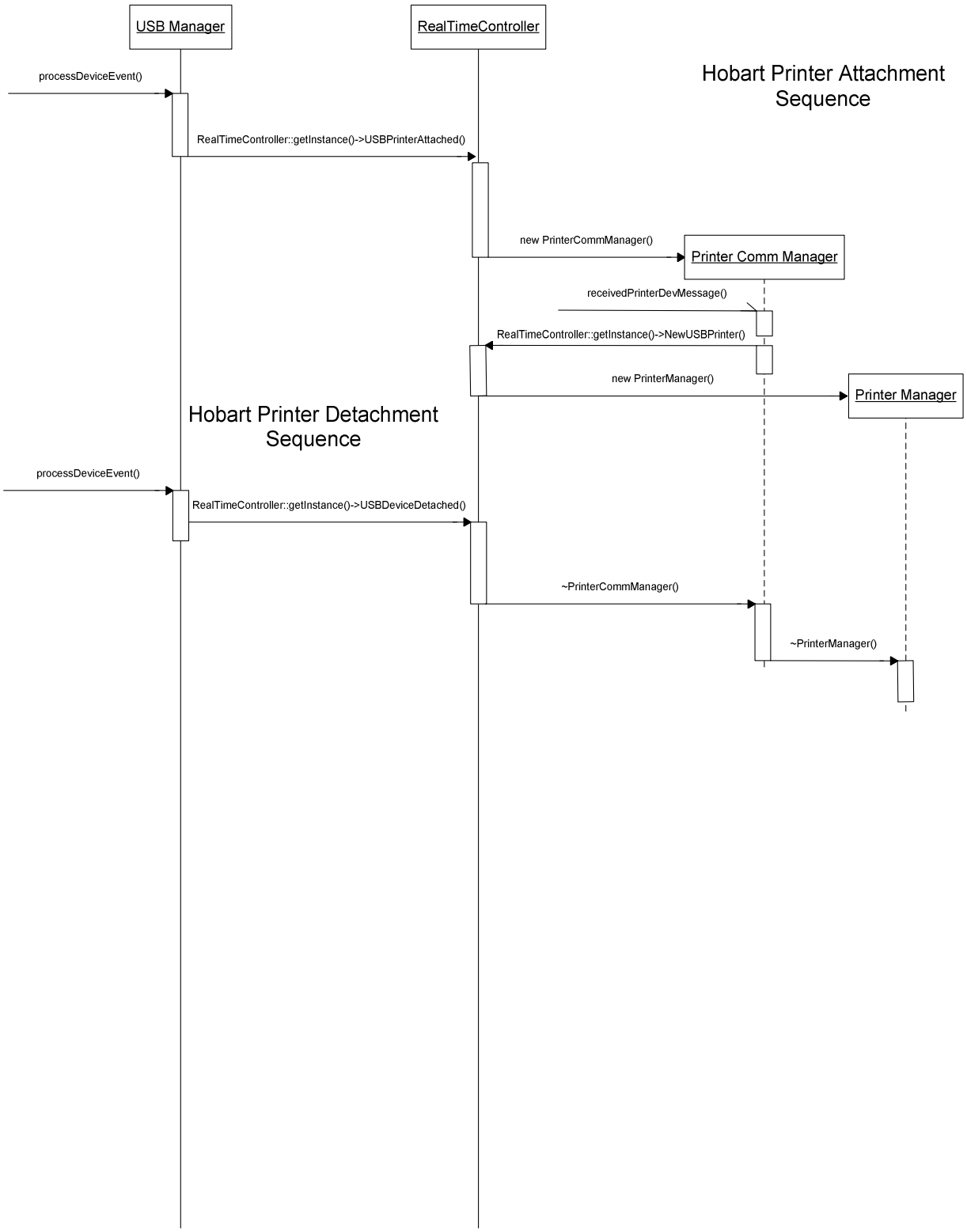


Figure 1.10 Hobart printer attachment sequence

6 Appendix

6.1 Name space used

_com_util
com
com::hobart
com::hobart::scale
com::hobart::scale::applet
com::hobart::scale::awt
com::hobart::scale::datastore
com::hobart::scale::datastore::mds
com::hobart::scale::graphic
com::hobart::scale::graphicviewer
com::hobart::scale::label
com::hobart::scale::labeleditor
com::hobart::scale::labeltext
com::hobart::scale::marqueeeditor
com::hobart::scale::menu
com::hobart::scale::mom
com::hobart::scale::nutrition
com::hobart::scale::parser
com::hobart::scale::swing
com::hobart::scale::swing::plaf
com::hobart::scale::swing::plaf::evolution
com::hobart::scale::swing::plaf::evolution::remote
com::hobart::scale::util
comm
edb
edb_adapter
email
hfc
Hobart
javax
javax::swing
MSXML
std
synchronization
Test
Util
util_acc
util_checksums
util_env
util_exception
util_filesystem
util_nibble
util_platform
util_registry
util_string
util_thread
util_time